RouteMind — Technical Report

AI-assisted, multi-agent delivery-route optimization for a food bank

Author: Poornesh Muppalam · Professor: Prashant Khanduri

Last-mile logistics | VRP with time windows | Full-stack

End-to-end technical overview — architecture, subsystems, development journey, engineering decisions, API contracts, configuration, and limitations.

1. Executive Summary

RouteMind plans a food bank's daily delivery van run. A dispatcher adds community delivery sites (by address search or map click), sets each site's **serving window** and **on-site drop time**, chooses when the van departs, and asks the system to build the shortest run that still reaches every site inside its window. The result is drawn on a live map as a turn-by-turn route with an estimated arrival (ETA) at each stop.

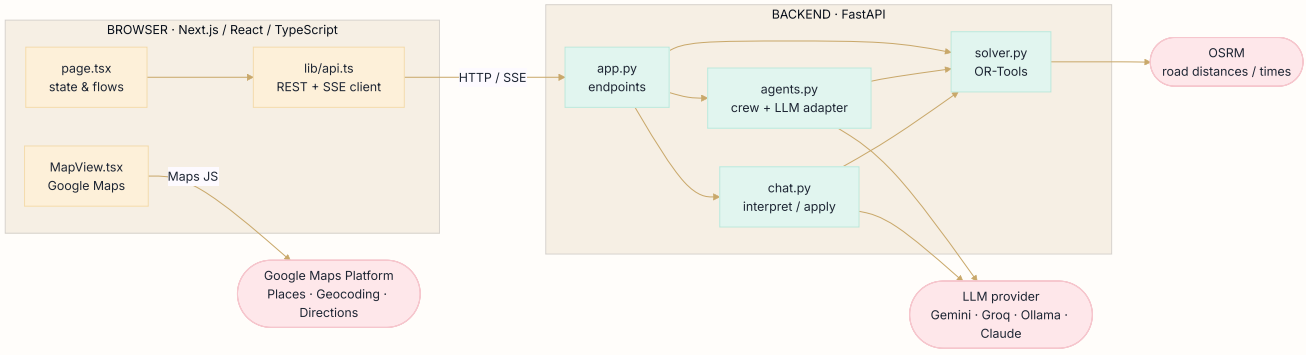
The core philosophy is a strict split: **deterministic math stays authoritative** and **language models do only what they are genuinely good at.**

- **Constraint solver (Google OR-Tools)** computes the actual visiting order, honoring time windows, drop times, departure time, and optional ordering pins. Road distances/times come from OSRM.
- **A crew of five agents (LangGraph)** wraps the solver with real analysis: input validation, pre-solve conflict prediction, solving, **auto-repair** of infeasible runs, and advice.
- **Natural-language chat** lets the dispatcher edit the run or ask questions; an LLM interprets (with a rule-based fallback) and the deterministic edits are applied + re-solved.

The app runs **free and offline-capable**: with no API key the chat uses a rule-based parser and agents use computed text; with a free LLM key (Google Gemini by default) the same flows gain natural-language quality.

2. System Architecture

FIGURE 1 — SYSTEM ARCHITECTURE & DATA FLOW



Technology stack

Layer	Technology
Frontend	Next.js 14 (App Router), React 18.3, TypeScript 5.5
Maps	`@react-google-maps/api` 2.20 — Maps JS, Places (Autocomplete), Geocoding, Directions, Geometry
Backend	FastAPI, Uvicorn, Pydantic 2

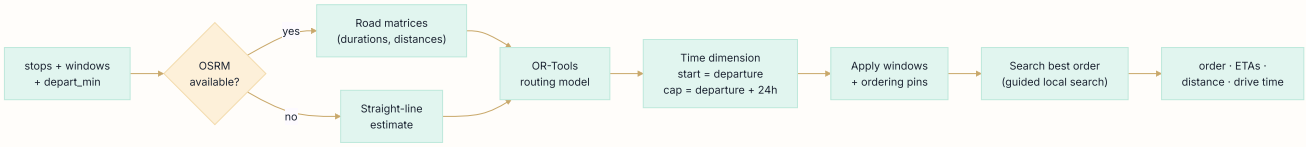
Optimization	Google OR-Tools (CP routing)
Routing data	OSRM public server + straight-line fallback
Agents	LangGraph (linear state graph)
LLM	OpenAI-compatible adapter — default Google Gemini `gemini-flash-lite-latest`; Anthropic fallback

3. Backend Subsystems

3.1 Solver (`solver.py`) — the honest core

- **Matrices:** `road_matrices()` fetches real driving durations/distances from OSRM (≤100 stops); on any failure, falls back to a haversine straight-line estimate. Reports `distance_source: road | estimate`.
- **OR-Tools model:** one vehicle, depot = node 0; arc cost = travel time + origin drop time.
- **Time dimension & departure:** start cumul fixed to `depart_min` (defaults to "now"); windows are range constraints; ETAs read back per stop.
- **Horizon fix:** max cumulative time = **departure + 24h** (not a midnight ceiling), so late/long runs don't falsely fail.
- **Pins:** "first/last" enforced on the Time dimension's cumulative vars.

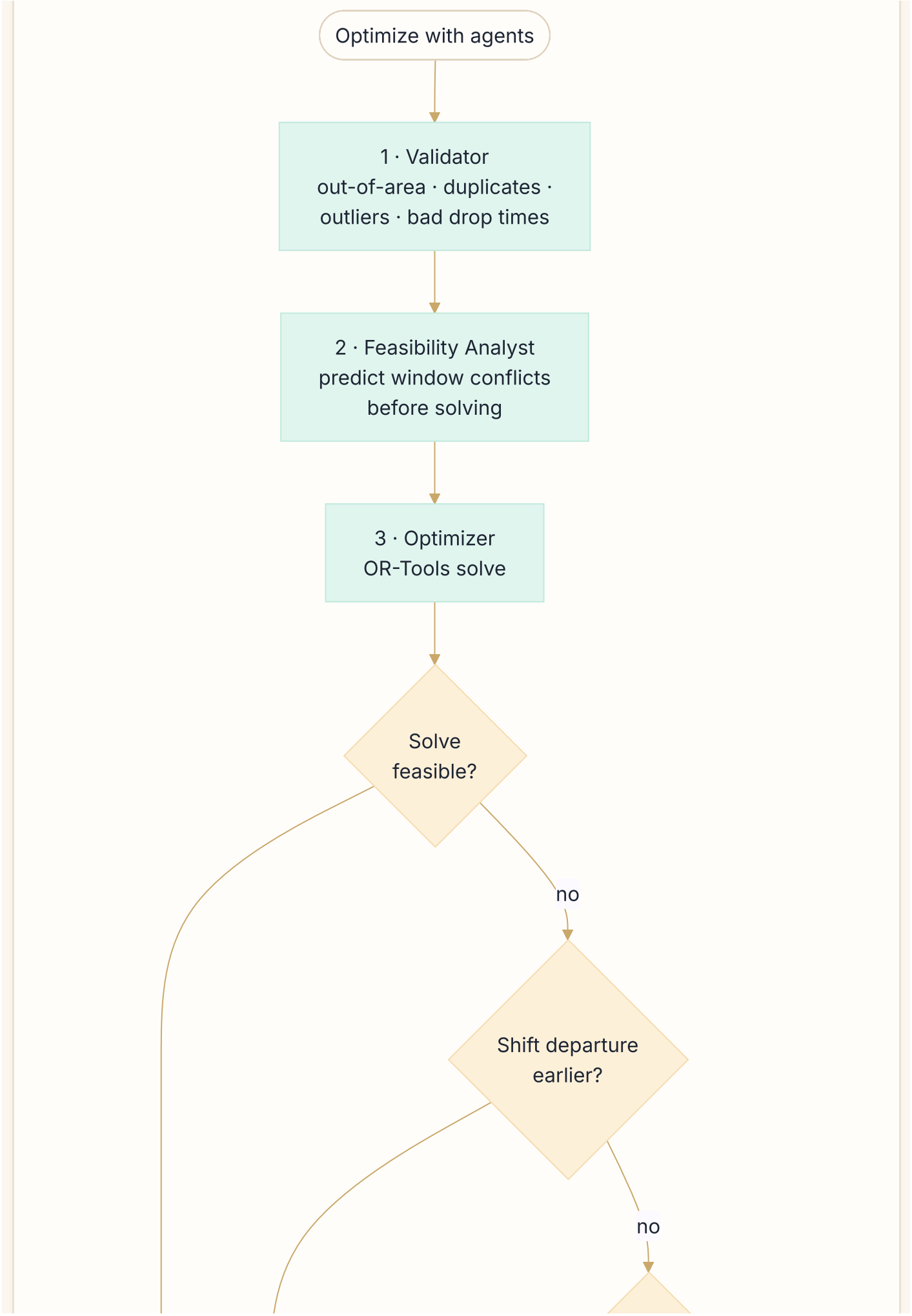
FIGURE 2 — SOLVE & ETA PIPELINE

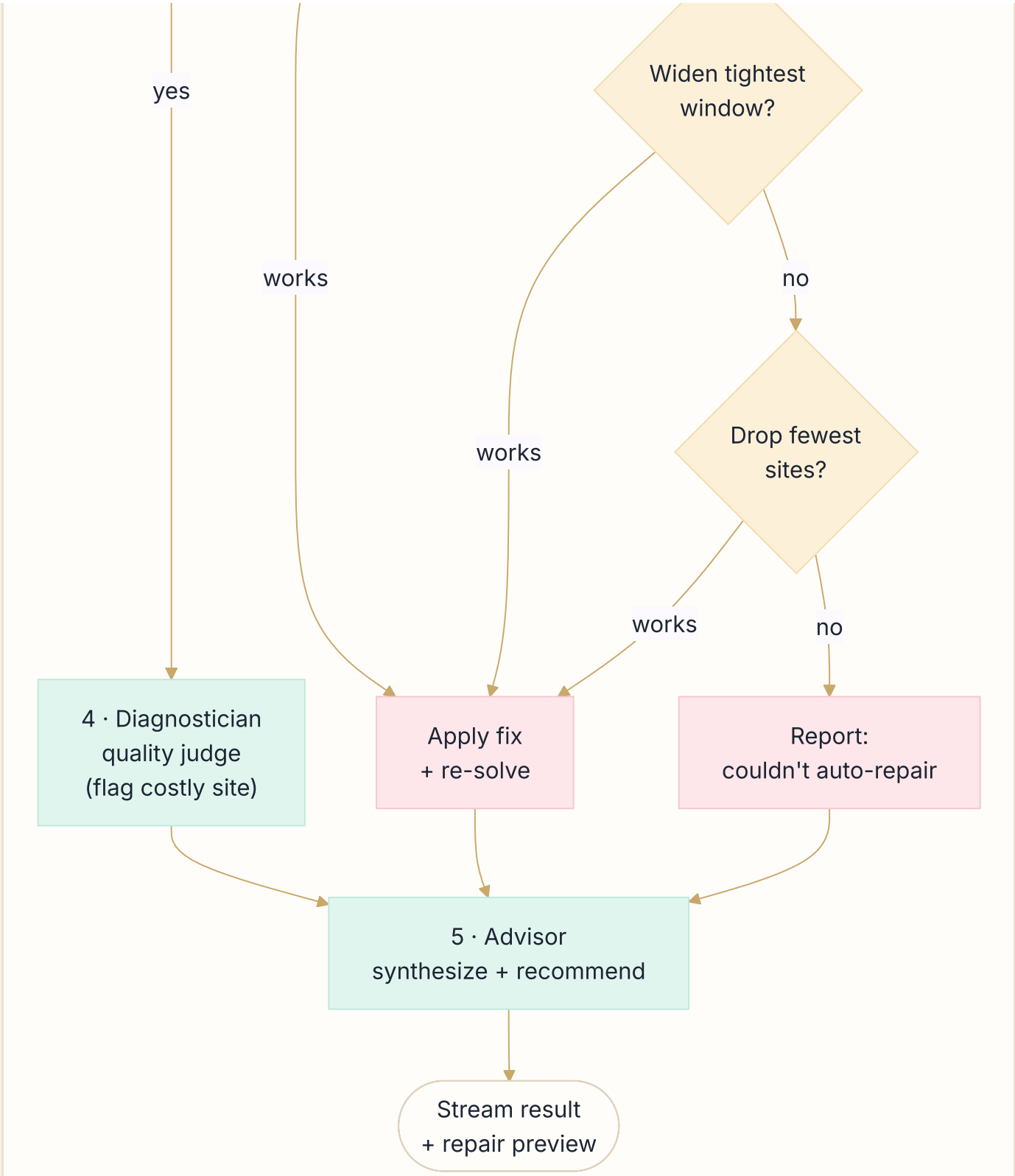


3.2 Agent crew (`agents.py`) — analysis, judgment, self-repair

A LangGraph linear pipeline; **each node is backed by computation** (works with no LLM key — the LLM only narrates). The Diagnostician is the standout: it **reacts** to failures.

FIGURE 3 — AGENT CREW & DIAGNOSTICIAN AUTO-REPAIR FIX LADDER

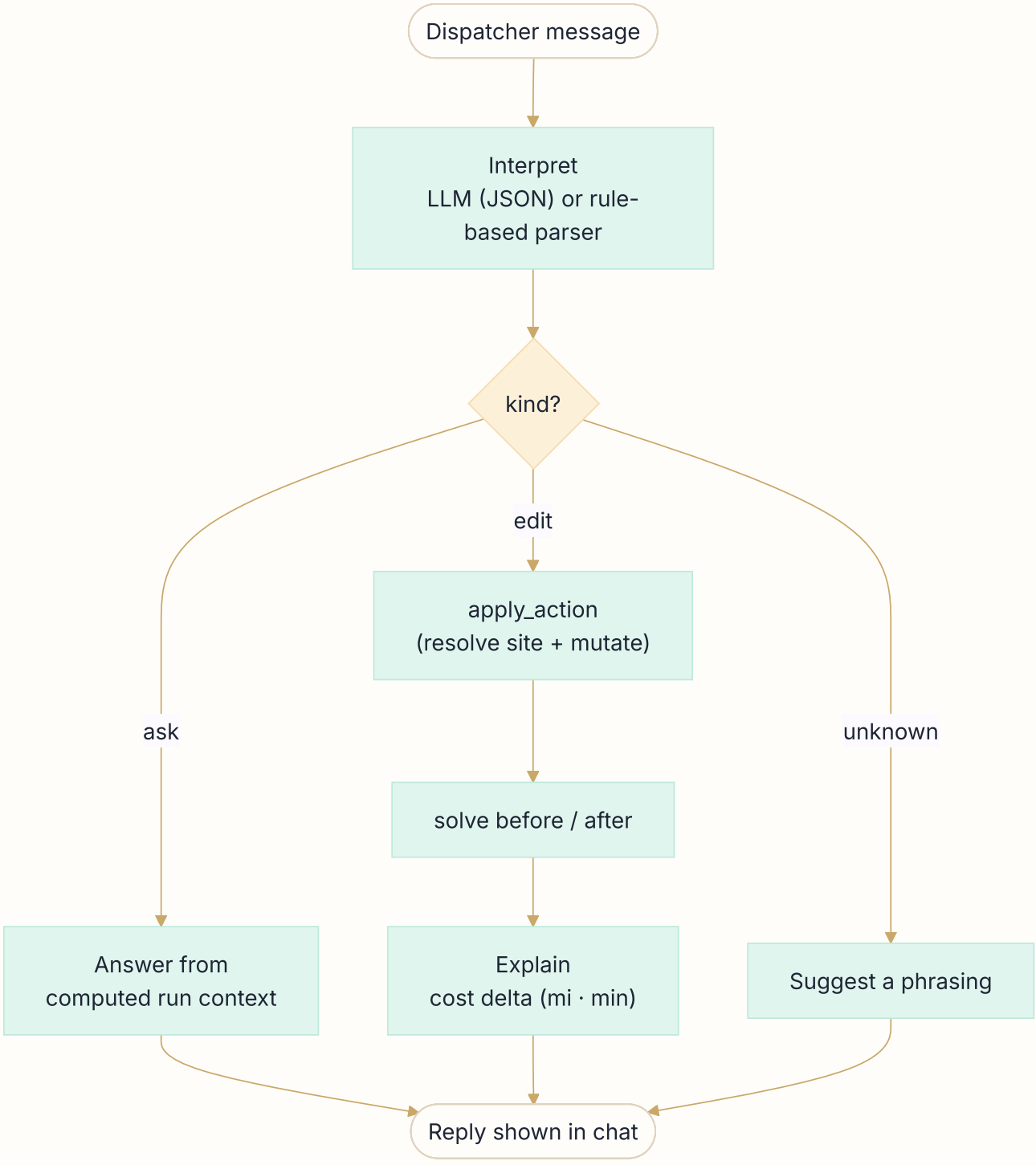

Optimize with agents
1 · Validator
out-of-area · duplicates ·
outliers · bad drop times
2 · Feasibility Analyst
predict window conflicts
before solving
3 · Optimizer
OR-Tools solve
Solve
feasible?
no
Shift departure
earlier?
no



LLM provider adapter. `_get_client()` resolves in priority order: OpenAI-compatible (`LLM_BASE_URL` + `LLM_API_KEY` → Groq / Gemini / Ollama / OpenAI), then Anthropic, else `None`. `_llm()` returns short text; `_llm_json()` returns a parsed object via JSON mode; `brain_info()` reports the active brain for the UI badge.

3.3 Chat (`chat.py`)

FIGURE 4 — CHAT TURN: INTERPRET → APPLY → RE-SOLVE / ANSWER



Edit kinds: `remove · set_window · set_service · reorder · set_return · reoptimize · ask · unknown`. An LLM path is used when a key is present; otherwise a regex parser handles times ("8 to 9 am") and site resolution by shared-word matching. Answers are always honest that live traffic is not modeled.

3.4 API (`app.py`)

Method & path	Purpose
`GET /`	Health check

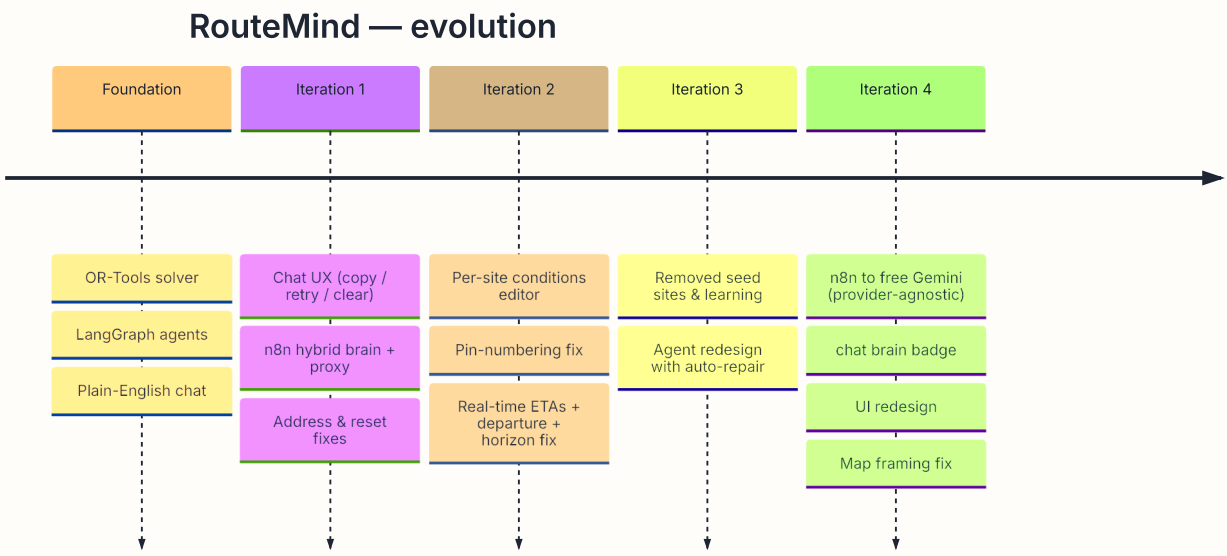
`POST /optimize`	Raw solve ("Solver only")
`POST /optimize/agents`	Run the crew once, return final state
`POST /optimize/agents/stream`	Stream crew steps over **SSE**; final event carries the (possibly repaired) result + `repair`
`POST /chat`	Full chat turn (interpret + apply / answer)
`GET /chat/info`	Active brain (Gemini / Groq / Claude / Ollama / Rules)
`POST /chat/apply`	Apply a pre-decided action + re-solve

4. Frontend Subsystems

- `page.tsx` — all app state and flows: add/remove/rename sites (service-area & duplicate guards), the ⚙ **per-site conditions editor** (drop time + serving window), the **"Van leaves at" departure** control, Optimize-with-agents (SSE → live panel + repair preview), Solver-only, and the chat (copy / retry / clear + brain badge).
- `MapView.tsx` — Google Maps via `useJsApiLoader`. Places Autocomplete with a **geocoding fallback** (Enter without picking a suggestion still adds the site); water-click rejection; markers numbered to match the run sheet (depot = "H"); Google-style route polyline + animated vehicle; `applyView()` framing + idle resize fix (prevents world-zoom / grey bands).
- `lib/api.ts` / `lib/types.ts` — typed client (`optimizeRoute`, `streamAgents` SSE reader, `sendChat`, `applyAction`, `getChatBrain`) and shared contracts.
- `globals.css` — warm/vibrant theme via CSS variables (cream + amber + coral + teal), Bricolage Grotesque headers, gradient wordmark; control panel on the left, map on the right.

5. Development Journey

FIGURE 5 — DEVELOPMENT TIMELINE



1. **Chat UX polish** — copy, retry-on-failure, clear chat.

2. **n8n hybrid brain (later removed)** — n8n interprets, backend applies via new `/chat/apply`; same-origin Next.js rewrite to bypass CORS; flexible response parser.
3. **Address search & reset fixes** — geocoding fallback; true reset-to-initial.
4. **Per-site conditions** — ⚙ editor for window + drop time on any added site.
5. **Pin numbering** — depot = "H", sites numbered from 1 in route order (matches run sheet).
6. **Real-time ETA model** — editable departure ("leave now"), always-on ETAs, departure+24h horizon.
7. **Scope trim** — removed default sites; removed the learning/unlearning feature.
8. **Agent redesign** — five real analysts + Diagnostician auto-repair + preview.
9. **n8n → free LLM** — provider-agnostic adapter, default Gemini; `/chat/info` + badge.
10. **UI redesign** — panel to the left; "Result" box removed with essentials relocated; warm theme.
11. **Map framing fix** — `applyView()` + idle resize + re-frame on stop changes.

---

6. Key Engineering Decisions & Trade-offs

- **Math vs. language separation** — OR-Tools owns correctness; LLMs never compute the route. Fast, reliable, auditable, and runnable with no LLM.
- **"Leave now" realism** — real ETAs can make morning windows infeasible in the afternoon (correct). The editable departure + auto-repair make this usable, not a dead end.
- **Agents that earn their place** — narration-only nodes replaced with ones whose decisions change outcomes.
- **Provider-agnostic LLM** — one OpenAI-compatible adapter for Gemini / Groq / Ollama / OpenAI, by env vars alone (no lock-in, no subscription).
- **Graceful degradation** — no OSRM → estimate; no LLM → rules/templates; unfixable run → honest message.

---

7. Configuration & Running

Backend (`backend/.env`):

```
LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai/
LLM_API_KEY=<gemini key>
LLM_MODEL=gemini-flash-lite-latest
OSRM_URL=https://router.project-osrm.org
```

Run: `cd backend && source .venv/bin/activate && uvicorn app:app --port 8000`

Frontend (`frontend/.env.local`):

```
NEXT_PUBLIC_BACKEND_URL=http://127.0.0.1:8000
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=<maps key, browser-only>
NEXT_PUBLIC_N8N_CHAT_URL=            # blank -> built-in /chat
```

Run: `cd frontend && npm run dev` → `http://localhost:3000`. Keys are separated by surface: the **Maps key is public** (browser, referrer-restricted); the **LLM key is server-side only**.

---

8. Known Limitations & Future Work

- **Single vehicle** — multi-van fleet routing not yet modeled (OR-Tools supports it).
- **No live traffic** — OSRM gives typical, not live, times; ETAs are honest about this.

- **Public OSRM limits** — rate-limited; production should self-host; Diagnostician re-solves are bounded to a few experiments.
- **Service area** — hard-coded to Northern California (≤130 mi of base).
- **Vestigial pieces** — `requirements.txt` still lists `scikit-learn` / `numpy` (removed learning feature); the `n8n/` files and flexible chat parser remain though unused — harmless.
- **Chat scope** — one action per turn; compound requests handled sequentially.

Next steps Multi-van fleet routing · driver mobile run sheet · a genuine "learn actual drop times from completed deliveries" loop · self-hosted OSRM · traffic-aware provider · one-click apply of chat-suggested fixes.

Prepared as an end-to-end technical overview of RouteMind. Behavior reflects the current codebase: backend `solver.py` / `agents.py` / `chat.py` / `app.py`; frontend `page.tsx` / `MapView.tsx` / `lib/*` / `globals.css`.